Palacan

Architettura Tecnica

Documento tecnico · versione 1.0 · 2026

Abstract

Questo documento descrive come Palacan è costruito: lo stack tecnologico, il modo in cui client e server si dividono le responsabilità, l'architettura del livello dati, e le scelte ingegneristiche che sostengono sia utenti individuali sia organizzazioni. Dove il discorso tocca i protocolli crittografici e il modello di minaccia, si rimanda al documento "Architettura di Sicurezza": qui l'attenzione è sull'ingegneria del sistema, non sulla crittografia in sé.

1. Stack tecnologico

Una panoramica dei componenti principali della piattaforma:

Componente	Tecnologia	Ruolo
Frontend	React + Vite	Interfaccia utente, esecuzione della crittografia lato client
Backend	Python, FastAPI	API REST, autenticazione, orchestrazione del routing dei messaggi
Database	PostgreSQL	Metadati operativi, utenti, workspace, ruoli
Migrazioni schema	Alembic	Versionamento controllato della struttura del database
Comunicazione real-time	WebSocket	Consegna istantanea e sincronizzazione multi-dispositivo
Crittografia	Olm / Megolm	End-to-end, eseguita sul dispositivo (dettagli nel whitepaper di Sicurezza)

La scelta di React e Vite per il frontend risponde a un'esigenza precisa: un'interfaccia reattiva, capace di gestire in tempo reale operazioni crittografiche sul thread principale senza percepibili rallentamenti per l'utente, mentre Vite garantisce tempi di build e di sviluppo rapidi anche su una base di codice che cresce.

Sul lato server, FastAPI è stato scelto per la sua natura asincrona nativa: gran parte del lavoro del backend — instradare messaggi, gestire connessioni WebSocket aperte, orchestrare notifiche verso più dispositivi contemporaneamente — è I/O-bound, ed è proprio lo scenario in cui un framework asincrono rende meglio rispetto a uno sincrono tradizionale.

2. Filosofia architetturale: separazione netta delle responsabilità

L'intero sistema è costruito attorno a un principio guida: il client fa tutto ciò che riguarda il significato dei dati, il server fa tutto ciò che riguarda il loro movimento. Il client genera le chiavi, cifra, decifra, gestisce le sessioni crittografiche con ogni interlocutore. Il server instrada, autentica, sincronizza, gestisce workspace e ruoli — ma non elabora mai il contenuto.

Questa separazione non è solo una scelta di sicurezza: è anche una scelta di ingegneria. Mantenere il server "cieco" rispetto al contenuto significa che la sua logica resta più semplice, più facile da scalare orizzontalmente, e meno esposta a classi intere di bug legati alla manipolazione di dati sensibili.

3. Livello dati

Il database PostgreSQL conserva esclusivamente ciò che serve al funzionamento del servizio: identità degli utenti, associazioni tra dispositivi e account, struttura dei workspace, ruoli e permessi, metadati di instradamento dei messaggi (mittente, destinatario, timestamp) — mai il contenuto dei messaggi in chiaro, che semplicemente non attraversa mai il server in quella forma.

Le migrazioni dello schema sono gestite con Alembic, che mantiene una cronologia versionata e reversibile di ogni modifica alla struttura del database: ogni evoluzione del modello dati — un nuovo campo, una nuova tabella, una relazione modificata — è tracciata, applicabile e annullabile in modo controllato, invece di essere il risultato di modifiche manuali dirette sull'ambiente di produzione.

4. Comunicazione in tempo reale

La consegna dei messaggi e degli aggiornamenti di stato avviene tramite connessioni WebSocket persistenti tra ogni dispositivo client e il server. A differenza di un modello a richieste ripetute (polling), la connessione WebSocket permette al server di notificare immediatamente un client nel momento esatto in cui c'è qualcosa da consegnargli, riducendo sia la latenza percepita sia il carico generato da richieste continue e in gran parte inutili.

Quando un messaggio cifrato arriva al server, questo lo instrada verso tutte le connessioni WebSocket attive associate ai dispositivi autorizzati del destinatario, mantenendo sincronizzato lo stato della conversazione su ciascuno di essi in tempo reale.

5. Architettura multi-dispositivo

Ogni dispositivo che un utente autorizza è trattato come un'entità indipendente all'interno del sistema, con una propria connessione, un proprio set di chiavi di sessione e una propria voce nel registro dei dispositivi associati all'account. Aggiungere un nuovo dispositivo, revocarne uno smarrito, o semplicemente disconnetterne uno temporaneamente inattivo sono operazioni isolate: non impattano lo stato degli altri dispositivi collegati allo stesso account.

Questo modello "a dispositivi paritari" — invece di un dispositivo "principale" e altri "secondari" — è stato scelto per rendere l'esperienza multi-dispositivo davvero simmetrica: qualunque dispositivo autorizzato ha pari accesso alle conversazioni, sincronizzate in tempo reale tramite lo stesso meccanismo WebSocket.

6. Architettura del workspace aziendale

Per l'uso aziendale, Palacan introduce il concetto di workspace: un livello logico che raggruppa utenti, ruoli e configurazioni sotto un'unica organizzazione, mantenuto come strato completamente separato dallo scambio dei contenuti. Un workspace ha una propria gerarchia di ruoli, una propria configurazione di sicurezza, un proprio insieme di dispositivi autorizzati — ma nessuna delle strutture dati del workspace contiene, in alcun punto, il contenuto delle conversazioni dei suoi membri.

L'architettura è inoltre pensata per il self-hosting: un'organizzazione può eseguire la propria istanza di Palacan sulla propria infrastruttura, mantenendo il controllo completo sull'ambiente di esecuzione, senza dipendere da un'infrastruttura condivisa gestita da terzi.

7. Funzionalità collaborative integrate

Oltre alla messaggistica, la piattaforma include strumenti collaborativi pensati per l'uso aziendale, come la gestione di attività (task) condivise all'interno di una conversazione. Dal punto di vista architetturale, questi strumenti seguono lo stesso modello del resto del sistema: lo stato collaborativo viene cifrato lato client e sincronizzato tramite lo stesso canale WebSocket usato per i messaggi, così che nessuna nuova superficie non cifrata venga introdotta nel sistema per il solo fatto di aggiungere una funzionalità diversa dal semplice testo.

8. Autenticazione e ciclo di vita delle sessioni

Ogni sessione utente è rappresentata da un token univoco, gestito lato server in modo da poter essere identificato, tracciato e revocato singolarmente — l'implementazione precisa di questo meccanismo è descritta nel whitepaper di Sicurezza. Dal punto di vista architetturale, ciò che conta è che l'autenticazione sia completamente disaccoppiata dalla logica di instradamento dei messaggi: un problema di sessione non ha mai la possibilità di esporre o compromettere il contenuto cifrato in transito.

9. Scalabilità

L'architettura è pensata per scalare orizzontalmente: poiché il server non mantiene stato relativo al contenuto delle conversazioni (che vive esclusivamente nei client), aggiungere istanze del backend per servire più utenti non introduce problemi di coerenza sui dati sensibili. Il collo di bottiglia naturale di un sistema di messaggistica — il numero di connessioni WebSocket simultanee — è gestito distribuendo le connessioni su più istanze del backend, con il database PostgreSQL come punto di coordinamento condiviso per gli stati che devono essere coerenti tra istanze (autenticazione, workspace, ruoli).

10. Conclusione

L'architettura descritta in questo documento nasce da una scelta di fondo: che l'ingegneria di un sistema di comunicazione debba riflettere gli stessi principi di rispetto per l'utente che il sistema promette a parole. Ogni componente — dal database che non vede i contenuti, al server che instrada senza comprendere, alla connessione real-time che sincronizza senza intercettare — è costruito perché quella promessa sia una conseguenza tecnica, non un impegno da mantenere a parte.

© 2026 Palacan · Documento tecnico, versione 1.0